COMS 3003A
Test 3

17 May, 2024

This is a closed book test.

Time allowance: 60 minutes

Total number of points: 50

1. For each of the following formulas, find a model where the formula is true and a model where the formula is false:

   (a) **(6 points)** $\forall x\, R(x,x)$;

   (b) **(6 points)** $\forall x \forall y \forall z\, (R(x,y) \wedge R(y,z) \to R(x,z)) \wedge \forall x \exists y R(x,y) \wedge \forall x\, \neg R(x,x)$.

2. **(10 points) Prove that the formula $\exists x\, (P(x) \vee Q(x)) \to \exists x\, P(x) \vee \exists x\, Q(x)$ is valid.**

   Suppose otherwise. Then, there exists a model $M = (D, I)$ and an assignment α in M such that $M \not\models \exists x\, (P(x) \vee Q(x)) \to \exists x\, P(x) \vee \exists x\, Q(x)[\alpha]$. Then,

   (1) $M \models \exists x\, (P(x) \vee Q(x))[\alpha]$.

   (2) $M \not\models \exists x\, P(x)[\alpha]$.

   (3) $M \not\models \exists x\, Q(x)[\alpha]$.

   By (1), there exists $\alpha' \stackrel{x}{=} \alpha$ such that

   (4) $M \models P(x) \vee Q(x)[\alpha']$,

   and so either

   (5a) $M \models P(x)[\alpha']$

   or

   (5b) $M \models Q(x)[\alpha']$.

But in either case, we obtain a contradiction: since $\alpha' \stackrel{x}{=} \alpha$, from (1) we obtain

(6a) $M \not\models P(x)[\alpha']$,

while from form (2) we obtain

(6b) $M \not\models Q(x)[\alpha']$.

3. **(6 points)** Prove that the formula $\forall x \exists y R(x, y) \to \exists y \forall x R(x, y)$ is not valid.

   Consider a model $M = (D, I)$ where $D = \{a, b\}$ and $I(R) = \{\langle a, b\rangle, \langle b, a\rangle\}$. Let α be an arbitrary assignment in M. Then, $M \models \forall x \exists y R(x, y)[\alpha]$, but $M \not\models \exists y \forall x R(x, y)[\alpha]$. Hence, $\forall x \exists y R(x, y) \to \exists y \forall x R(x, y)$ is not valid.

4. In the proof of undecidability of the first-order logic, we describe computations of Turing machines on the empty string ϵ. When describing a Turing machine $M = (Q, \Sigma, \Gamma, q_0, q_1, q_2, \delta)$, we use the following vocabulary (remember that every first-order language contains the equality symbol $=$):

   - the constant symbol 0 ('zero');
   - the unary function symbol S ('successor' on natural numbers);
   - for every $k \in \{0, \ldots, |Q|\}$, a unary predicate letter $Q_k(x)$ ('at step x, the machine M is in state q_k');
   - a binary predicate letter $C(x, y)$ ('at step x, the machine M scans cell y');
   - for every $k \in \{0, \ldots, |\Gamma|\}$, a binary predicate letter $S_k(x, y)$ ('at step x, cell y contains symbol s_k').

   When describing computations of Turing machines, we will be making the following assumptions:

   - the tape is one-way infinite (hence the cells can be numbered 0, 1, 2, etc.);
   - machines can move left and right (no other types of head movement are allowed);
   - the left-most cell, i.e., cell 0, always contains an end-of-tape marker, which is a special tape symbol;
   - states are always called q_0, q_1, q_2, etc.;
   - the starting state is q_0, the accepting state is q_1, and the rejecting state is q_2;
   - the tape symbols are always called s_0, s_1, s_2, etc.
   - s_0 is the blank;
   - s_1 is the end-of-tape marker.

   Write formulas saying the following, in the vocabulary listed above:

   (a) **(4 points)** A formula saying that M halts.

   $$\exists x\, (Q_1(x) \lor Q_2(x)).$$

(b) **(4 points)** A formula describing the initial configuration of M on the empty word ϵ.

$$C(0,0) \wedge Q_0(0) \wedge S_1(0,0) \wedge \forall x\,(\neg(x=0) \to S_0(0,x)).$$

(c) **(5 points)** Suppose that $q_i s_k \to q_j s_l R$ is an instruction from δ. Write a formula describing what happens when M executes this instruction.

$$\forall x \forall y \Bigg(Q_i(x) \wedge S_k(x,y) \wedge C(x,y) \to$$
$$\big(Q_j(S(x)) \wedge S_l(S(x),y) \wedge C(S(x),S(y)) \wedge$$
$$\forall z\,(\neg(z=y) \to \bigwedge_{k=0}^{|\Gamma|-1} (S_k(x,z) \to S_k(S(x),z)))))\Bigg).$$

5. (a) **(5 points) Suppose there exists a Turing machine M_V that decides first-order validity (i.e., it can determine, for every first-order formula φ, whether φ is valid on not). Describe a Turing machine M_ϵ that decides termination of Turing machines on the empty string (use a diagram for your description).**

(b) **(2 points) Does the machine M_V described in part (a) exist? Explain your answer.**

Such a machine does not exists. If it existed, we could construct, as we have seen in (a), a machine deciding a machine deciding the undecidable problem of determining whether a Turing machine terminates on the empty string.

(c) **(2 points) Is it possible to write a computer program that determines whether a first-order formula is valid or not? Expain your answer.**

No. If such a program existed, it could be converted into a Turing machine solving the same problem, but we have just seen that such a machine does not exist.